

Ruminations on reflection and access

Abstract

This paper explains a couple of problems with the current library API in P2996. In particular, that API is problematic because it provides no way to express the same access controls as the language does. At all, not just "in a convenient form". That API also (or because of that) makes it very difficult and tedious to write reflective programs that obey the usual access controls.

This is not at all a good situation. The vast majority, a really vast majority of reflection use cases can be expressed with constructs that obey the usual access controls. Including, but not limited to, memberwise hashing and things like that. If we do reflection and injection right.

"Full access"

Let's begin by looking at the notion of "full access", which means "everything everywhere is accessible".

Let's define a couple of example types:

```
class PolarRep {
    double z;
    double phi;
public:
    PolarRep(double x, double y);
    double x();
    double y();
    double z();
};

class VectorRep {
    double x;
    double y;
public:
    VectorRep(double x, double y);
    double x();
    double y();
    double z();
};

class C1 {
    double x;
    double y;
public:
    C1(double x, double y);
    double x();
    double y();
    double z();
};

class C2 {
```

```

    double z;
    double phi;
public:
    C2(double x, double y);
    double x();
    double y();
    double z();
};

class C3 {
    PolarRep rep;
public:
    C3(double x, double y);
    double x();
    double y();
    double z();
};

class C4 {
    VectorRep rep;
public:
    C4(double x, double y);
    double x();
    double y();
    double z();
};

```

Now let's use those types in a composite:

```

class X {
    C1 c1;
    C2 c2;
    C3 c3;
    C4 c4;
public:
    X(double x, double y);
    double x();
    double y();
    double z();
    friend void fiddle(X&);
};

```

Okay then. In what context of X does the suggested full access exist? Outside it, in the external user context of X? In its non-static member functions? In its class definition? In the friend function? In the constructor?

The correct answer is Nowhere. There is no "full access" in C++. It's not a thing. The constructor, non-static member functions, the friend function, and the class definition of X have full access to all members of X - but not full access to the members of those members. There is no "basis operation" that starts with full access and then builds restrictions on top of it.

Metadata access vs. data access

There are suggestions that it's enough to access-control data, and leave metadata (types, names, cardinalities) fully-accessible.

That suggestion also doesn't model how the language works. Access controls control metadata as well as data. You can't name the data members of PolarRep/VectorRep/C1/C2/C3/C4, you can't get their types, you can't count how many such members exist; you do not have access to the metadata any more than you have access to the data.

And of course you don't. Those are implementation details. You are not allowed to form untoward dependencies to them that would cause trouble if any of those things mentioned are modified, the types, the names, the cardinalities. The use of those can be limited to particular access scopes, and refactorings that change those things incompatibly are done with the knowledge that the refactoring can break only the scopes that have sufficient access, and nothing else, so nothing else needs to be considered when performing such refactorings.

But hold my beverage, there's more to it.

Protected metadata access vs. data access

Let's define another helper:

```
class Base {
protected:
    C1 c1;
    C2 c2;
    C3 c3;
    C4 c4;
    struct Foo {
        int a, b;
    };
public:
    Base(double x, double y);
    double x();
    double y();
    double z();
};
```

Let's also define a composite that uses it:

```
class D : public Base {
public:
    D(double x, double y);
    double x();
    double y();
    double z();
};
```

Here we have a rather more interesting situation. D has full metadata access to Base, except for things that are metadata of something D doesn't have full access to.

For example, the protected non-static members. D doesn't have full access to them. D has access to them only through a D*/D&/D. If D is somehow given a B*/B&/B, it can't access those protected non-static data members, or their types, or their cardinalities. It can access them through this, because the type of it is D*.

And still, that non-non-static metadata like struct Foo isn't fully accessible here either. Only B and D have access to it, nobody else does.

Access to virtual functions

Let's define another helper:

```
class BaseInterface {
private:
    virtual void do_op();
    virtual void do_op2();
    virtual void do_op3();
    virtual void do_op4();
public:
```

```

    // these call the private virtuals
    void op();
    void op2();
    void op3();
    void op4();
};

```

And then use it:

```

class Concrete : public BaseInterface {
    void do_op() override;
    void do_op2() override;
    void do_op3() override;
    void do_op4() override;
};

```

This case is interesting; Concrete must have enough metadata access to be able to utter the same return and parameter types as the BaseInterface functions, in order to be able to override them. But other than that, it has no access whatsoever to the declarations of the private virtuals in BaseInterface, except for an ephemeral "I can override it, and I can know when that's correct."

A suggestion for where this should lead

Simply, we should have

1. a query facility that returns only the accessible members (it then obeys access controls for both metadata and data)
2. a query facility that returns accessible members when accessed with a particular type (this is for correct protected access)
3. a query facility that returns only the metadata of overrideable virtual functions (not the data; you can't call inaccessible virtuals, so this facility should give you just enough information to generate overrides)

Let's settle that hash

The hashing example is roughly as follows, using the metaclass notation:

```

class(breakthrough_memberwise_hash) UserFoo {
private:
    what ever;
    data to;
    be hashed;
    goes here;
public:
    // whatever public API
};

template <class T> size_t some_generic_breakthrough_hasher(const T& t) {
    if type has opted in to breakthrough-memberwise-hashing,
    grab its non-static data members regardless of access
    and inject/splice a hash of those here
}

```

An access-obeying form of that is instead

```

class(memberwise_hash) UserBar { // original source form
private:
    what ever;
    data to;
    be hashed;
}

```

```

    goes here;
public:
    // whatever public API
};

template <class T> size_t some_generic_hasher(const T& t) {
    return memberwise_hash(t);
}

```

Now, what needs to happen here is that the metaclass-like metaprogram on UserBar is treated by the language so that it takes the user-written UserBar as a "protoproject", and generates, via injection, roughly the following:

```

class UserBar { // generated form after 'metaclass' transformation
private:
    what ever;
    data to;
    be hashed;
    goes here;
    friend size_t memberwise_hash(const UserBar& ub) {
        grab the accessible non-static data members
        and inject/splice a hash of those here
    }
public:
    // whatever public API
};

```

For some of us, that some_generic_breakthrough_hasher is everything we feared would happen, when discussing these matters already a decade ago in SG7. And we have consistently worked towards such things being done like the some_generic_hasher does it, instead.

There are certainly cases where you have code that you can't modify to make it inject friends for its classes. But then you can't opt in such types to memberwise operations by adding a tag on a class either. And it remains highly questionable how generic and unconstrained the access-breaking operations on such types should be, as opposed to perhaps being rare exceptions that operate on concrete types, rather than being templates that may end up being used with any type, legacy or not.

"But this approach means the query for accessible members returns different things in different contexts, that's error-prone"

It's far more error-prone (and far more brittle, and scalability-, maintainability-, and refactoring-unfriendly) to gain access to something you're not supposed to. It's perfectly natural that such a query returns different values in different contexts, that's how the language works. Different things are accessible in different contexts.

Different results in different contexts are a feature, not a bug. What's important is that readers and maintainers of the code can instantly know by looking at such a query that it will never return things that are inaccessible. Which turns into knowledge that access controls aren't bypassed. The basis operation returns only things that are accessible, and that result can then be filtered further. Code that is messing with private data members or private member functions is placed in member functions or friend functions, or in helper functions that receive a "delegated" access context. Contexts that do not have access to the privates can't mess with them.

We could of course entertain the alternative that instead of the query not returning inaccessible things, it would be an error if it would try to. As far as I can see, that doesn't change the API signature of the query, just its behavior. The problem is, however, that then it's again very difficult to provide just the right filters to the query so that it doesn't try to return inaccessible things. Access controls in C++ are non-trivial, and the P2996's API approach of telling users to build them themselves on top of a basis operation that doesn't actually exist in the language makes it very hard to get them right. So would emitting an error from a query that's supposed to give you what you have access to.